Guia-de-resolucion-del-examen-De...

Anónimo

Introducción a la Ingeniería del Software y los Sistemas de Información II

2º Grado en Ingeniería Informática - Ingeniería del Software

Escuela Técnica Superior de Ingeniería Informática Universidad de Sevilla

Accede al documento original

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Guía de resolución del examen — DeliverUS Orders

Requisitos funcionales a implementar

RF.01 – Visualización de pedidos por restaurante Como propietario, quiero ver un listado con todos los pedidos realizados a uno de mis restaurantes. Para cada pedido, necesito conocer su fecha de creación, el estado, la dirección de entrega y el precio, para poder hacer un seguimiento y gestión de los mismos.

RF.02 – Edición de algunas propiedades de pedidos Como propietario, quiero poder editar la dirección de entrega y el precio de los pedidos realizados a mis restaurantes, para corregir errores o actualizar información relevante.

Pruebas de aceptación RF.02

- El propietario puede acceder a la edición de un pedido desde el listado de pedidos de su restaurante.
- Solo se muestran los campos dirección de entrega y precio total y han de presentarse con los valores actuales del pedido que se desea editar.
- Ambos campos (dirección de entrega y precio total) son obligatorios.
- Si se intenta guardar un pedido sin completar la dirección de entrega, se muestra un mensaje de error indicando que el campo es obligatorio.
- Si se intenta guardar un pedido sin establecer el precio total, se muestra un mensaje de error indicando que el campo es obligatorio.
- Si se introduce un precio igual o inferior a 0, se muestra un mensaje de error indicando que el valor debe ser mayor que 0.
- Al guardar correctamente los cambios, se actualiza la información del pedido y se muestra un mensaje de confirmación.
- Tras la correcta actualización de datos, se ha de navegar automáticamente hacia el listado de pedidos que debe presentar la información actualizada.

RF.03 – Visualización de analíticas de pedidos de restaurante Como propietario, quiero visualizar analíticas sobre los pedidos de cada uno de mis restaurantes: el importe facturado hoy, el número de pedidos en estado pending, el número de pedidos de hoy en estado delivered y el número de pedidos realizados ayer.

RF.04 – Cambio de estado de un pedido Como propietario, quiero poder cambiar el estado actual de los pedidos realizados a mis restaurantes. Desde pending puedo pasarlo a in process; de in process a sent; y de sent a delivered.

Pruebas de aceptación RF.04

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

- Al cambiar correctamente el estado de un pedido, se ha de reflejar en el listado y se ha de actualizar la sección de analíticas. Además se ha de mostrar un mensaje de éxito.
- Si no se ha podido cambiar el estado se ha de mostrar un mensaje de error.
- Los pedidos en estado `delivered` no pueden cambiar de estado ni se debe presentar la opción de cambiarlo.

EJERCICIO 1 — Listado de pedidos · 3 puntos

Fichero: `OrdersScreen.js`

Aspectos dados por el enunciado:

- La pantalla es accesible desde el botón *Check orders* en `RestaurantDetailScreen`.
- La función `getOrderImage(status)` ya está proporcionada en el mismo fichero.
- La función `getRestaurantOrders(restaurantId)` ya está implementada en `RestaurantEndpoints.js`.

¿Qué se pide?

Mostrar una lista de pedidos de un restaurante, con imagen de estado, fecha, estado, dirección y precio.

Conceptos clave

`useState`

Guarda datos que, cuando cambian, repintan la pantalla automáticamente. El fichero base ya tenía `restaurant` con `useState`. Nosotros necesitamos añadir un nuevo estado para los pedidos:

```js
const [orders, setOrders] = useState([])
```

- `orders` → el dato en memoria (el array de pedidos)
- `setOrders` → la acción que actualiza el estado y provoca un re-render
- `[]` → el valor inicial (empieza vacío, ya que se carga desde el backend)

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

`useEffect`

Ejecuta código como efecto secundario. Estructura base:

```js
useEffect(() => {
  // código a ejecutar cuando el hook se activa
}, [dep1, dep2]) // array de dependencias
```

Dependencia	Comportamiento
`[]`	Se ejecuta una sola vez al montar el componente
`[obj1, obj2]`	Se ejecuta cuando cambia alguna dependencia
(sin array)	Se ejecuta en cada re-render

En el código base ya existe un `useEffect` con dependencia `[route]` para cargar el detalle del restaurante. Se dispara cuando cambia `route`, es decir, al entrar a la pantalla por primera vez. **Añadimos las llamadas a fetchRestaurantOrders y fetchRestaurantAnalytics dentro de ese mismo useEffect**, aprovechando que `route` también cambiará cuando volvamos de `EditOrderScreen`:

```js
useEffect(() => {
  fetchRestaurantDetail()
  fetchRestaurantOrders()    // Ejercicio 1
  fetchRestaurantAnalytics() // Ejercicio 3
}, [route])
```

> ⚠️ Cuando el usuario edita un pedido y vuelve a `OrdersScreen`, le pasamos `{ dirty: true }` (ver Ejercicio 2). Eso cambia el objeto `route`, lo que dispara este `useEffect` y recarga la lista con los datos actualizados.

`fetchRestaurantOrders`

Es una función `async` porque `getRestaurantOrders` devuelve una Promise: la petición HTTP al backend tarda un tiempo desconocido. Sin `await`, `setOrders` recibiría una Promise sin resolver

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

en lugar de los datos reales. El bloque `try/catch` captura cualquier error de red o del servidor y lo muestra al usuario con `showMessage`.

```js
const fetchRestaurantOrders = async () => {
  try {
    const fetchedOrders = await getRestaurantOrders(route.params.id)
    setOrders(fetchedOrders)
  } catch (error) {
    showMessage({
      message: `There was an error while retrieving orders. ${error}`,
      type: 'error',
      style: GlobalStyles.flashStyle,
      titleStyle: GlobalStyles.flashTextStyle
    })
  }
}
```

`FlatList`

El `return` del código base solo tenía `<></>`. Lo reemplazamos con una `FlatList` porque el enunciado pide un **listado** (RF.01: *"ver un listado con todos los pedidos"*). Siempre que el enunciado diga **listado**, **lista** o **mostrar todos los X**, la estructura que corresponde es una `FlatList`.

Propiedades que usamos:

Prop	Para qué sirve
`data`	El array de pedidos (estado `orders`)
`renderItem`	Función que pinta cada pedido
`ListHeaderComponent`	Cabecera (imagen del restaurante + analíticas) que hace scroll junto a la lista
`ListEmptyComponent`	Mensaje cuando no hay pedidos
`keyExtractor`	Clave única por elemento (obligatorio en FlatList)

```jsx
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```jsx
return (
  <View style={styles.container}>
    <FlatList
      ListHeaderComponent={renderHeader}
      ListEmptyComponent={renderEmptyOrdersList}
      style={styles.container}
      data={orders}
      renderItem={renderOrder}
      keyExtractor={item => item.id.toString()}
    />
  </View>
)
```

`renderOrder`

Es el corazón visual del listado. Usa el componente `ImageCard` (ya existente en el proyecto), pasándole la imagen del estado con `getOrderImage(status)`:

```jsx
const renderOrder = ({ item }) => {
  return (
    <ImageCard
      imageUri={getOrderImage(item.status)}
      title={'Order created at ' + item.createdAt}
    >
      <TextRegular>Status: {item.status}</TextRegular>
      <TextRegular>Address: {item.address}</TextRegular>
      <TextSemiBold>{item.price?.toFixed(2)}€</TextSemiBold>
      <View style={styles.actionButtonsContainer}>
        {/* Botones Edit (Ej.2) y Advance (Ej.4) van aquí */}
      </View>
    </ImageCard>
  )
}
```

> `toFixed(2)` es una función de JavaScript que formatea el número a exactamente 2 decimales.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

EJERCICIO 2 — Edición de pedido · 3 puntos

Ficheros: `EditOrderScreen.js` + `RestaurantsStack.js` + botón en `OrdersScreen.js`

¿Qué se pide?

Formulario para editar `address` y `price` de un pedido existente, con validación y navegación de vuelta.

Conceptos clave

`useState` en `EditOrderScreen`

Necesitamos tres estados:

```js
const [order, setOrder] = useState({})
const [initialOrderValues, setInitialOrderValues] = useState({ address: null, price:
const [backendErrors, setBackendErrors] = useState()
```

- `order` → guarda el pedido completo cargado del backend. Lo necesitamos en estado (no en variable local) para que `order.id` siga disponible cuando el usuario pulse Save, aunque el componente se haya repintado mientras tanto. Se inicializa con `{}` porque el fetch es asíncrono.
- `initialOrderValues` → los valores con los que se pre-rellenan los campos del formulario. Formik necesita este objeto desde el primer render, aunque los valores reales lleguen después. Se inicializa con `null` en ambos campos; cuando el `useEffect` recibe los datos del backend llama a `setInitialOrderValues` para actualizarlos.
- `backendErrors` → errores devueltos por el servidor, distintos de los errores de validación local que gestiona yup. Se inicializa sin valor (undefined) para que la condición `backendErrors && ...` no renderice nada hasta que haya errores reales.

Patrón Formik + yup

Todo el proyecto usa este patrón para formularios. Formik gestiona el estado del formulario y yup define las reglas de validación. Hay que reconocerlo porque `EditOrderScreen` ya tiene los imports de ambas librerías en el código base:

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```js
import * as yup from 'yup'
import { Formik } from 'formik'
```

`validationSchema`

El esquema de validación traduce directamente las pruebas de aceptación del enunciado:

```js
const validationSchema = yup.object().shape({
  address: yup.string().required('Address is required'),
  price: yup
    .number()
    .positive('Price must be greater than 0') // cubre el caso <= 0
    .required('Price is required')
})
```

Reglas yup más comunes:

Regla yup	Significado
`.required('msg')`	Campo obligatorio
`.string()` / `.number()`	Tipo esperado
`.max(n, 'msg')`	Longitud máxima (strings)
`.positive('msg')`	Solo números positivos (> 0)
`.email('msg')`	Formato de email válido
`.url('msg')`	Formato de URL válida
`.nullable()`	Permite que el campo sea null

`useEffect` para cargar el pedido

Seguimos el mismo patrón del Lab 5 y Lab 8: función `async` interna dentro del `useEffect`, porque `useEffect` no puede ser `async` directamente.

```js
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
useEffect(() => {
  async function fetchOrder () {
    try {
      const fetchedOrder = await getById(route.params.id)
      setOrder(fetchedOrder)
      setInitialOrderValues({ address: fetchedOrder.address, price: fetchedOrder.pri
    } catch (error) {
      showMessage({
        message: `There was an error while retrieving order details. ${error}`,
        type: 'error',
        style: GlobalStyles.flashStyle,
        titleStyle: GlobalStyles.flashTextStyle
      })
    }
  }
  fetchOrder()
}, [route])
```

`route.params.id` viene del `navigation.navigate('EditOrderScreen', { id: item.id })` de `OrdersScreen`. Solo guardamos en `initialOrderValues` los dos campos editables (`address` y `price`); Formik los usará para pre-rellenar los `InputItem`.

`updateOrder`

Es la función que Formik llama en `onSubmit` **solo si yup ha validado correctamente** el formulario. Por eso no necesitamos validar manualmente aquí. Recibe `values`, que es el objeto con los valores actuales del formulario: `{ address, price }`.

Flujo completo:

1. El usuario pulsa Save → Formik llama a `handleSubmit`
2. `handleSubmit` ejecuta la validación con yup
3. Si hay errores → los muestra en pantalla y **no** llama a `updateOrder`
4. Si todo es válido → llama a `updateOrder(values)`
5. `updateOrder` hace PUT al backend con `update(order.id, values)`
6. Si el backend responde OK → muestra mensaje de éxito y navega a `OrdersScreen`
7. Si el backend responde con error → guarda los errores en `backendErrors` para mostrarlos

```
js
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
const updateOrder = async (values) => {
  setBackendErrors([])
  try {
    await update(order.id, values)
    showMessage({
      message: 'Order updated successfully',
      type: 'success',
      style: GlobalStyles.flashStyle,
      titleStyle: GlobalStyles.flashTextStyle
    })
    // Pasamos { dirty: true } para que route cambie en OrdersScreen
    // y su useEffect([route]) se dispare, recargando la lista actualizada
    navigation.goBack()
  } catch (error) {
    setBackendErrors(error.errors)
    showMessage({
      message: `There was an error while updating the order. ${error}`,
      type: 'error',
      style: GlobalStyles.flashStyle,
      titleStyle: GlobalStyles.flashTextStyle
    })
  }
}
```

El componente `<Formik>` en el return

Formik actúa como gestor de estado del formulario. Envuelve todo el contenido del return y se encarga de guardar los valores de cada campo, ejecutar la validación con yup cuando el usuario pulsa Save, y llamar a `updateOrder` solo si todo es válido.

Sus propiedades clave:

Prop	Para qué sirve
`validationSchema`	Las reglas yup que definen qué es un valor válido
`initialValues`	Con qué valores aparecen los campos al montar el componente
`enableReinitialize`	Sin esto, Formik solo lee `initialValues` la primera vez que monta (cuando aún son `null`). Con esto, detecta que `setInitialOrderValues` los ha actualizado y repinta los campos con los datos reales del pedido
`onSubmit`	La función a ejecutar cuando `handleSubmit` es llamado **y** la validación pasa

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Dentro de Formik se usa el patrón de render prop `{({ handleSubmit }) => ...}`. `handleSubmit` es lo que se pasa al `onPress` del botón Save, **no** `updateOrder` directamente:

```jsx
return (
  <Formik
    validationSchema={validationSchema}
    initialValues={initialOrderValues}
    enableReinitialize={true}
    onSubmit={updateOrder}
  >
    {({ handleSubmit }) => (
      <ScrollView>
        <View style={{ padding: 20 }}>

          <InputItem name='address' label='Address:' />
          <InputItem name='price' label='Price:' keyboardType='decimal-pad' />

          {backendErrors &&
            backendErrors.map((error, index) => (
              <TextError key={index}>{error.msg}</TextError>
            ))
          }

          <Pressable onPress={handleSubmit} style={...}>
            <MaterialCommunityIcons name='content-save' color={'white'} size={20} />
            <TextRegular textStyle={styles.text}> Save</TextRegular>
          </Pressable>

        </View>
      </ScrollView>
    )}
  </Formik>
)
```

> `InputItem` ya está integrado con Formik: usa el `name` para leer y escribir en el estado del formulario, y muestra el error de yup automáticamente si el campo no pasa la validación. El `name` debe coincidir exactamente con la clave en `initialValues`.

Registrar la pantalla en `RestaurantsStack.js`

El stack ya importaba `EditOrderScreen` pero le faltaba registrarla como pantalla navegable. Sin esto, `navigation.navigate('EditOrderScreen', ...)` lanza un error en tiempo de ejecución.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Se añade:

```jsx
<Stack.Screen
  name='EditOrderScreen'
  component={EditOrderScreen}
  options={{ title: 'Edit order' }}
/>
```

> El string del `name` debe coincidir **exactamente** con el usado en `navigate()`.

Botón de acceso en `OrdersScreen.js`

Dentro de `renderOrder`, añadimos un `Pressable` en el `actionButtonsContainer`:

```jsx
<Pressable
  onPress={() => navigation.navigate('EditOrderScreen', { id: item.id })}
  style={({ pressed }) => [
    {
      backgroundColor: pressed
        ? GlobalStyles.brandBlueTap
        : GlobalStyles.brandBlue
    },
    styles.actionButton
  ]}
>
  <View style={{ flex: 1, flexDirection: 'row', justifyContent: 'center' }}>
    <MaterialCommunityIcons name='pencil' color={'white'} size={20} />
    <TextRegular textStyle={styles.text}> Edit</TextRegular>
  </View>
</Pressable>
```

EJERCICIO 3 — Analíticas · 2 puntos

Ficheros: `OrdersScreen.js` + `RestaurantEndpoints.js`

¿Qué se pide?

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Mostrar 4 métricas del restaurante en una cuadrícula 2×2 con el estilo ya proporcionado.

Conceptos clave

`useState` para analytics

Hay que asegurarse de que el objeto de analíticas comienza en `null` para poder hacer el renderizado condicional (mostrar `'-'` mientras los datos no han llegado aún). Se añade al inicio de `OrdersScreen`:

```js
const [analytics, setAnalytics] = useState(null)
```

Se inicializa con `null` (no con `{}`) para distinguir "todavía no ha llegado" de "llegó pero está vacío". Eso permite el operador ternario: `analytics ? analytics.invoicedToday : '-'`.

Llamada GET en `RestaurantEndpoints.js`

La función `getRestaurantAnalytics` estaba vacía. Solo hay que añadir la llamada al endpoint correcto, siguiendo el mismo patrón que `getRestaurantOrders`:

```js
function getRestaurantAnalytics (restaurantId) {
  return get(`/restaurants/${restaurantId}/analytics`)
}
```

`fetchRestaurantAnalytics` en `OrdersScreen.js`

La estructura es idéntica a `fetchRestaurantOrders`: función `async` con `try/catch`, `await` para esperar la respuesta y `setAnalytics` para guardar el resultado en estado.

El objeto que devuelve el backend (estructura dada en el enunciado):

```js
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
{
  restaurantId: 1,
  numYesterdayOrders: 2,
  numPendingOrders: 1,
  numDeliveredTodayOrders: 1,
  invoicedToday: 65.0
}
```

```js
const fetchRestaurantAnalytics = async () => {
  try {
    const fetchedAnalytics = await getRestaurantAnalytics(route.params.id)
    setAnalytics(fetchedAnalytics)
  } catch (error) {
    showMessage({
      message: `There was an error while retrieving analytics. ${error}`,
      type: 'error',
      style: GlobalStyles.flashStyle,
      titleStyle: GlobalStyles.flashTextStyle
    })
  }
}
```

Propiedades Flex

En `OrdersScreen`, en el apartado `styles`, ya existían los estilos `analyticsContainer`, `analyticsRow` y `analyticsCell`, pero `analyticsRow` estaba **vacío**. El enunciado pide completar las propiedades relacionadas con Flex para conseguir la cuadrícula 2×2 de la Figura 1.

```js
analyticsRow: {
  flexDirection: 'row',        // las dos celdas se colocan en horizontal
  justifyContent: 'space-around' // espacio uniforme entre ellas
},
analyticsCell: {
  alignItems: 'center', // centra el texto dentro de cada celda
  flex: 1,              // cada celda ocupa exactamente la mitad del ancho
  // ... resto de estilos ya dados (colores, sombra, etc.)
}
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

- Sin `flexDirection: 'row'` las celdas se apilan verticalmente (comportamiento por defecto).
- Sin `flex: 1` las celdas no se distribuyen equitativamente.

Renderizar en pantalla

La función `renderAnalytics` ya está dada en el código base. Solo hay que llamarla dentro de `renderHeader`, justo debajo del `ImageBackground`, para que las analíticas aparezcan debajo de la imagen del restaurante y hagan scroll junto a la lista:

```jsx
const renderHeader = () => {
  return (
    <View>
      <ImageBackground ...>
        <View style={styles.restaurantHeaderContainer}>
          {/* ... contenido del restaurante ya dado ... */}
        </View>
      </ImageBackground>
      {/* Ej.3: añadimos la llamada a renderAnalytics */}
      {renderAnalytics()}
    </View>
  )
}
```

EJERCICIO 4 — Cambio de estado · 2 puntos

Fichero: `OrdersScreen.js`

¿Qué se pide?

Botón "Advance" en cada pedido que avanza su estado. Los pedidos `delivered` no deben tener este botón.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Conceptos clave

Llamar a `nextStatus`

La función `nextStatus(order)` ya está implementada en `OrderEndpoints.js` y hace la llamada correcta al backend según el estado actual del pedido. Solo hay que llamarla desde `handleNextStatus`, que en el código base está vacía.

Tras el éxito hay que recargar **tanto** la lista de pedidos **como** las analíticas, porque la prueba de aceptación del RF.04 exige que ambas se actualicen al cambiar el estado:

```js
const handleNextStatus = async (order) => {
  try {
    await nextStatus(order)
    showMessage({
      message: 'Order status updated successfully',
      type: 'success',
      style: GlobalStyles.flashStyle,
      titleStyle: GlobalStyles.flashTextStyle
    })
    // Recargar lista y analíticas para reflejar el cambio en pantalla
    fetchRestaurantOrders()
    fetchRestaurantAnalytics()
  } catch (error) {
    showMessage({
      message: `There was an error updating the order status. ${error}`,
      type: 'error',
      style: GlobalStyles.flashStyle,
      titleStyle: GlobalStyles.flashTextStyle
    })
  }
}
```

Renderizado condicional del botón

La prueba de aceptación dice que los pedidos `delivered` no deben mostrar la opción de cambiar estado. Esto se hace dentro de `renderOrder` en `OrdersScreen.js`.

Se usa `&&` para renderizar condicionalmente: si la condición es `false`, no se renderiza nada.

```jsx
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
{item.status !== 'delivered' && (
  <Pressable
    onPress={() => handleNextStatus(item)}
    style={({ pressed }) => [
      {
        backgroundColor: pressed
          ? GlobalStyles.brandGreenTap
          : GlobalStyles.brandGreen
      },
      styles.actionButton
    ]}
  >
    <View style={{ flex: 1, flexDirection: 'row', justifyContent: 'center' }}>
      <MaterialCommunityIcons name='skip-next' color={'white'} size={20} />
      <TextRegular textStyle={styles.text}> Advance</TextRegular>
    </View>
  </Pressable>
)}
```

> El nombre del icono `skip-next` está especificado en el enunciado. El icono viene de `MaterialCommunityIcons`.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.